SalinDugo System Documentation

Prediction, Import, and Print Report Modules

Document version: 1.0 **Prepared for:** Thesis Defense **System:** SalinDugo — Blood Demand Forecasting and Inventory Planning Platform

---

Table of Contents

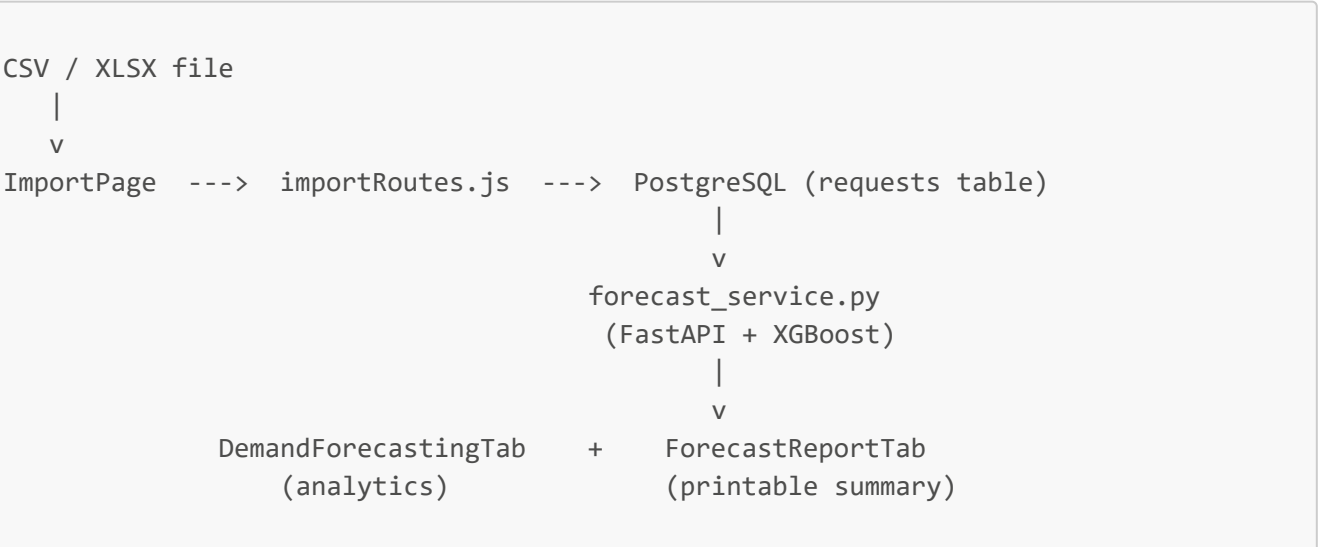
---

1. System Overview

SalinDugo is a three-tier system that helps hospitals plan blood inventory by forecasting future blood demand for each blood type. The three modules covered in this document are:

Module	Purpose	File(s)
Import	Bulk-loads historical blood request data into the database	`ImportPage.jsx`, `importRoutes.js`
Prediction	Generates 30-day demand forecasts and visualizes them with KPIs/charts	`DemandForecastingTab.jsx`, `forecast_service.py`
Print Report	Compiles a printable A4 PDF-style executive summary	`ForecastReportTab.jsx`

The data flow at a glance:



2. The Import Process

2.1 Why Import Matters

Forecasting is only as accurate as its history. The Import module is the **single entry point** that turns spreadsheets into the structured time-series the model needs. Without it, no historical data exists for the XGBoost model to learn lag and rolling-window features from.

2.2 Frontend — `ImportPage.jsx`

Step 1: File Selection The user selects a `.csv` or `.xlsx` file. The component reads the first 6 lines using `selected.text()` and stores them in `preview` so the user can visually verify the file before uploading.

Step 2: Required Schema The file must contain exactly these columns:

- `request_date` (YYYY-MM-DD)
- `blood_type` (A+, A-, B+, B-, AB+, AB-, O+, O-)
- `units_needed` (integer ≥ 0)
- `status` (open / matched / fulfilled / cancelled)

Step 3: Template Download `handleDownloadTemplate()` generates a CSV blob in-memory and triggers a browser download. This guarantees the user starts from a correctly formatted template.

Step 4: Upload `handleUpload()` wraps the file in a `FormData` object and POSTs it to `/api/import/requests` with `multipart/form-data`. The auth token (set globally in the axios instance) authenticates the hospital.

2.3 Backend — `importRoutes.js`

Step 1: Authentication The route is protected by `authenticateToken`. `getHospitalId(req)` returns `req.user.id` so every imported row is automatically scoped to the logged-in hospital.

Step 2: Multer Buffering `multer.memoryStorage()` keeps the file in RAM (limit: 2 MB) instead of writing to disk — faster and more secure.

Step 3: Polymorphic Parsing — `parseFile()`

- For CSV/text: splits by newline, auto-detects tab vs comma delimiter, and constructs row objects keyed by header.
- For XLSX: uses ExcelJS to load the workbook and iterate `sheet.eachRow()`.

Step 4: Per-Row Validation Loop For each row the backend:

1. Parses `request_date` (handles Excel serial numbers via `(value - 25569) * 86400 * 1000`).
2. Normalizes `blood_type` to uppercase and validates against `VALID_TYPES`.
3. Trims `units_needed`, rejects empty strings, allows `0` (important — zero-demand days are valid features).
4. Validates `status` against `VALID_STATUS`.
5. Pushes parameterized values into a single bulk-insert array.

Step 5: Atomic Bulk Insert The entire batch is wrapped in `BEGIN ... COMMIT`. On any failure the transaction is rolled back, so the database is never left in a partial state. This is critical because the model retrains on whatever rows are present — partial imports would silently degrade forecast accuracy.

Step 6: Response On success the API returns `{ message, rows_inserted }`. The frontend then resets the form and shows a green success alert.

---

3. The Prediction Process — Frontend (`DemandForecastingTab`)

This tab is the **operational dashboard**. It calls the FastAPI forecasting service, then computes KPIs and chart-ready data structures using `useMemo` for performance.

3.1 Data Fetching

When `hospitalId` or `mode` changes, four endpoints are hit in parallel for live mode:

Endpoint	Stored in	Purpose
`/api/history-total/:id`	`historyTotal`	Daily summed history (for YoY comparison)
`/api/history/:id`	`historyByType`	Per-blood-type history (for trend baselines)
`/api/forecast-total/:id?days=30`	`totalForecast`	30-day total predicted demand
`/api/forecast/:id?days=30`	`bloodTypeForecast`	30-day predicted demand per blood type with CI bands

For backtest mode a single call to `/api/backtest/:id` returns merged actual-vs-predicted data plus error metrics.

3.2 KPI Cards (Top of Page)

30-Day Total Demand

- **Formula:** `Σ totalForecast[i].total_predicted_demand`
- **Why it matters:** A single number that tells operations how many units in total they should be prepared to dispense over the planning horizon. Drives bulk-procurement decisions.

Peak Daily Demand

- **Formula:** `max(totalForecast[i].total_predicted_demand)`
- **Why it matters:** Identifies the worst-case day so the hospital can pre-stage stock. Inventory is planned around peaks, not averages.

Top Blood Type (30 Days)

- **Formula:** Group `bloodTypeForecast` by `blood_type`, sum `predicted_demand`, take the type with the highest total.

- **Why it matters:** Highlights the single most strained blood type for the month. Procurement and donation drives can be prioritized accordingly.

Top Blood Type (7 Days)

- **Formula:** Same as above but only summing the **first 7 forecast days** per type.
- **Why it matters:** Catches short-term demand surges that a 30-day average would dilute. A type can be quiet for the month but urgent for the week.

Year-over-Year Demand

- **Formula:**
  - `forecastTotal = Σ totalForecast.total_predicted_demand`
  - `lastYearTotal = Σ historyTotal[d].blood_requests` where `d` falls within the same calendar window shifted -1 year.
  - `yoyChange = ((forecastTotal − lastYearTotal) / lastYearTotal) × 100`
- **Tone:**
  - `> 5%` → red ("Increasing")
  - `< -5%` → green ("Decreasing")
  - otherwise → "Stable"
- **Why it matters:** Anchors the forecast against a real-world historical baseline so managers can ask "is this normal?" — guards against silently overconfident predictions.

3.3 Chart-by-Chart Explanation (Live Mode)

(a) Total Demand Forecast — `AreaChart`

- **Data:** `totalForecast` directly.
- **Computation:** None on the frontend — pure render.
- **Why:** Visualizes the trajectory of overall demand so managers can see if demand is climbing, plateauing, or declining over the next month.

(b) Demand Change — `BarChart`

- **Data:** `accelerationData`
- **Formula:** `change[i] = forecast[i] − forecast[i−1]` (first day = 0).
- **Why:** Day-over-day deltas reveal volatility and turning points. A flat total trend can still hide sharp swings — this chart surfaces them.

(c) 30-Day Demand Budget by Blood Type — `DemandBudgetCard`

Built from `useDemandBudget`:

- `demandBudget30Days[bt]` = sum of first 30 forecast days per blood type.
- `rankedBudget` = entries sorted descending.
- Inside the card:
  - `share = (total / totalAll) × 100`
  - `daily = total / 30`

    - `priority` is bucketed by share: `≥30%` High, `≥15%` Medium, otherwise Low.
    - `level` combines priority **and** trend: High-priority + Increasing/New = **Critical**.
- **Why:** Translates raw numbers into actionable priority labels. "Critical" is the cue for emergency procurement, not just a high number.

(d) Demand by Blood Type — `LineChart`

- **Data:** `groupedData` — pivot of `bloodTypeForecast` keyed by date, with one column per blood type.
- **Why:** Lets users compare relative demand patterns across blood types simultaneously. Reveals whether spikes are isolated (one type) or systemic (all types).

(e) High-Demand Streak Alert — `useHighDemandStreak`

- **Baseline:** Mean of the last 14 actual days per blood type.
- **Streak detection:** Walks the next 30 forecast days; counts consecutive days where `forecast > baseline`. Records streaks of length **≥ 3**.
- **Why:** Sustained pressure (3+ days in a row above normal) is a stronger signal than a single peak. Persistent streaks indicate a structural shift, not noise.

(f) 7-Day Demand Trend by Blood Type — `useBloodTypeTrend`

- `past` = avg of last 7 actual days per blood type.
- `future` = avg of first 7 forecast days per blood type.
- `changePct = (future - past) / past × 100`
- **Trend classification:**
    - `> +20%` → Increasing
    - `+5% to +20%` → Slight ↑
    - `-5% to +5%` → Stable
    - `-20% to -5%` → Slight ↓
    - `< -20%` → Decreasing
    - `past = 0, future > 0` → New Demand
    - `past = 0, future = 0` → No Demand
- **Why:** Bridges the gap between recent reality and near-future forecast, giving a directional signal that managers can interpret without reading the chart.

(g) Forecast Confidence Intervals — `ComposedChart`

- **Data:** `forecastIntervalData` — derived from `bloodTypeForecast` for the selected blood type.
- **Computation:** `ci_80_lower / upper` and `ci_95_lower / upper` come from the backend (residual-based bounds — see §5.10). The frontend clips lower bounds at 0 (negative demand is meaningless).
- **Why:** Point forecasts hide uncertainty. The 80% / 95% bands let planners see the range of plausible outcomes — wider bands = lower confidence = more buffer stock needed.

3.4 Backtest Mode

Backtest mode validates the model's accuracy on a held-out window (Oct 1–31, 2025). It renders:

Total Demand Backtest — `ComposedChart`

- **Green line:** actual demand (truth)
- **Red line:** model prediction
- **Green band:** 80% CI; **Blue band:** 95% CI
- **Metrics shown below:**
  - **RMSE (Model / Baseline):** Root Mean Squared Error. Penalizes large misses heavily. Lower = better.
  - **MAE (Model / Baseline):** Mean Absolute Error. Average miss in units. Easy to interpret operationally.
  - **MAPE (Model / Baseline):** Mean Absolute Percentage Error. Scale-free, useful for comparing across blood types.
  - **Actual vs Predicted totals + Difference + % Error:** Bias check — is the model systematically over- or under-forecasting?
- **Why all of these?** Each metric answers a different question. RMSE catches catastrophic misses; MAE describes the typical day; MAPE allows cross-type comparison; total bias reveals systemic skew. The **baseline** comparison (lag-1: "tomorrow = today") is the honesty check: a model that beats lag-1 is genuinely learning patterns.

Backtest by Blood Type

Same chart and metrics, filtered to one blood type at a time. This is essential because aggregate metrics can hide that the model is great for O+ and terrible for AB-.

---

4. The Print Report Process

4.1 File: `ForecastReportTab.jsx`

This tab generates a **print-friendly executive summary** designed to be exported as a PDF via the browser's print dialog (`window.print()`).

4.2 How the Print Layout Works

The component uses scoped CSS:

```
@media print {
  body * { visibility: hidden !important; }
  .print-area, .print-area * { visibility: visible !important; }
}
```

Only elements inside `.print-area` are visible when printed — the toolbar, navigation, and "Print Report" button (`.no-print`) are hidden. Page breaks are manually controlled with `.print-page-break` between long sections.

4.3 Computed Sections

Executive Summary

A natural-language paragraph composed from `summary` (totals, peak, min, std), `topBT` (most-demanded type), `summary.highDays` (count of days >120% of average), and `yoy` (year-over-year change). This is the single most important section — decision-makers read it first.

Key Metrics Cards

Six-card grid covering: Total Demand, Avg Daily, Peak Day, Lowest Day, Top Blood Type, and YoY Change. Each metric is sourced from the same `summary` and `yoy` memos used in the dashboard, ensuring consistency between the on-screen view and the printed report.

Total Demand Forecast Chart

A simple line chart of `totalChartData` — daily predicted units. Animations are disabled (`isAnimationActive={false}`) so the chart renders immediately for printing.

30-Day Demand by Blood Type Bar Chart

Sourced from `bloodTypeBreakdown` — total units summed per blood type, sorted descending. Each bar is colored using the `BT_COLORS` map for visual distinction.

Average Demand by Day of Week

- **Computation:** `dayOfWeekData` buckets each forecast day by `getDay()` (0=Sun … 6=Sat) and computes the average per weekday.
- **Why:** Reveals weekly seasonality. Hospitals can schedule staffing and donations around the busiest weekdays.

Forecast Trend by Blood Type

A pivoted line chart (`bloodTypePivoted`) showing every blood type as its own line over time. Useful for visually identifying which types drive overall demand.

Detailed Blood Type Breakdown Table

Columns: Rank, Blood Type, Total, Avg/Day, Peak, Peak Date, % of Total. Computed from `bloodTypeBreakdown`. Provides the auditable numeric backing for the charts.

Top 5 Highest Demand Days Table

- Sorts `totalForecast` descending by predicted demand and takes the top 5.
- For each day computes `diffPct = (total − avgDaily) / avgDaily × 100` to show how far above average it is.
- **Why:** Tells planners exactly which days to prepare for, with weekday context (e.g., "Mon, Mar 15 — +34% vs avg").

4.4 Printing Workflow

1. User clicks **Print Report**.
2. `window.print()` fires — the browser opens its print dialog.

3. The user picks "Save as PDF" or sends to a physical printer.
4. The styled `.print-area` is rendered as A4 with 1.2 cm margins.

---

5. The Forecasting Engine — `forecast_service.py` (Detailed Walkthrough)

This is the heart of the system. This section is the most important for the thesis defense — it documents how the model is loaded, how features are computed, how predictions are made, and how confidence is quantified.

5.1 Service Architecture

`forecast_service.py` is a **FastAPI** application that exposes six HTTP endpoints. It is the bridge between the trained XGBoost model artifacts (`xgboost_real.pkl`) and the React frontend.

```
React (axios)  ---HTTP--->  FastAPI  ---SQL--->  PostgreSQL
                              |
                              v
                        XGBoost Model
                      (per blood type:
                       classifier + regressor)
```

CORS is enabled for `http://localhost:5173` (dev) and the production Vercel URL.

5.2 Constants You Must Understand First

Constant	Value	Meaning
`TARGET`	`"blood_requests"`	The column the model predicts
`BACKTEST_CUTOFF_DATE`	2025-09-30	Last day of training data for the backtest
`BACKTEST_START_DATE`	2025-10-01	First day of the held-out test window
`BACKTEST_END_DATE`	2025-10-31	Last day of the held-out test window
`FEATURES`	(list of 24)	Exact feature order the model was trained on

Why the explicit feature list matters: XGBoost requires features to be passed in the **same order** they were trained on. Any drift between training and inference breaks predictions silently. The list is hard-coded as a contract between training and serving.

5.3 Model Loading

```
models = joblib.load("model/xgboost_real.pkl")
```

`models` is a **dictionary** keyed by blood type. For each blood type it stores two sub-models:

- `models[bt]["clf"]` — a binary classifier predicting *whether* there will be demand on that day.
- `models[bt]["reg"]` — a regressor predicting *how much* demand if any.

This **two-stage architecture** is important: blood request data is full of zeros (many days have no requests for a given type). A pure regressor would be biased toward zero. The classifier-then-regressor design lets the system handle sparse, intermittent demand correctly.

5.4 Step 1 — Load Raw History

```
def load_center_history(hospital_id):
    SELECT DATE(request_date) AS date,
           blood_type,
           SUM(units_needed) AS blood_requests
    FROM requests
    WHERE hospital_id = %s
      AND (status IS NULL OR status != 'cancelled')
    GROUP BY DATE(request_date), blood_type
```

- Aggregates all non-cancelled requests by date + blood type.
- Cancelled requests are excluded — they did not represent real demand.
- The result has gaps (days with zero requests are missing entirely).

5.5 Step 2 — Fill Gaps with Zeros

```
def complete_missing_dates(df):
    for bt in df["blood_type"].unique():
        full_range = pd.date_range(min, max, freq="D")
        sub = sub.set_index("date").reindex(full_range)
        sub["blood_requests"] = sub["blood_requests"].fillna(0)
```

Why this matters: Lag features (`lag_1`, `lag_7`, etc.) only work if every date is present. A missing day would make `lag_7` actually mean "7 *records* ago" instead of "7 *days* ago" — corrupting all features. Filling with zero is correct because *no record = no demand*.

5.6 Step 3 — Build Features (`build_features`)

For each blood type, sorted chronologically, the function computes 24 features. Grouped by purpose:

Lag Features (autocorrelation)

- `lag_1, lag_2, lag_3, lag_7, lag_14` — demand 1, 2, 3, 7, 14 days ago.
- **Why:** Blood demand is highly autocorrelated. Yesterday and last week are strong predictors of tomorrow.

Difference Features (momentum)

- `diff_1 = lag_1 - lag_2` — short-term momentum (rising or falling vs the day before).

- `diff_7 = lag_7 - lag_14` — week-over-week momentum.
- **Why:** Captures trend direction independent of absolute level.

Rolling Statistics (smoothed signals)

- `roll_mean_3, roll_mean_7` — rolling averages.
- `roll_std_7, roll_std_14` — rolling standard deviations (volatility).
- `roll_max_7, roll_max_14` — recent peak demand.
- `ewm_7` — exponentially weighted mean (recent days weighted more).
- **Why:** Raw lags are noisy. Rolling stats smooth out noise and give the model a sense of current "regime" (high-volatility vs steady).
- **Critical detail:** Rolls are computed on `shifted = g[TARGET].shift(1)` — they always exclude the current day, preventing **target leakage**.

Sparsity Features (zero-handling)

- `is_zero_lag1` — flag: was yesterday zero?
- `zero_streak` — how many consecutive previous days were zero?
- **Why:** Blood demand is intermittent. A long zero-streak is a strong signal that today is also likely zero. The classifier uses this heavily.

Spike Features (anomaly handling)

- `spike_flag = 1` if `lag_1 > 2 × roll_mean_7`.
- **Why:** Lets the model react to recent unusual surges differently from steady-state demand.

Calendar / Seasonality Features

- `weekday, month, is_weekend` — categorical date components.
- `sin_week, cos_week, sin_month, cos_month` — **cyclical encoding** of weekday and month.
- **Why use sin/cos?** Sunday (6) and Monday (0) are adjacent, but a raw integer encoding makes them seem far apart. Encoding as `(sin, cos)` on a unit circle preserves cyclicality, so the model sees Sun→Mon as close.

After feature construction, `.dropna()` removes the first ~14 rows where lags are undefined.

5.7 Step 4 — Compute Features for One Future Day (`compute_row_features`)

This is the **inference-time mirror** of `build_features`, but for a single day with a `hist` series that includes prior predictions. It produces the same 24 features so the trained model can score the day.

Key implementation notes:

- `tail_3.mean()`, `tail_7.mean()`, etc. compute rolling stats from the most recent N predicted/actual values.
- `zero_streak` walks backward from the most recent value, counting consecutive zeros.
- Calendar features are computed from `new_date` directly.
- All features are returned as a Python dict, then wrapped in a one-row DataFrame and reordered to match `FEATURES`.

5.8 Step 5 — The Two-Stage Prediction

```
prob = float(clf.predict_proba(X_future)[0][1])   # P(demand > 0)
reg_pred = float(reg.predict(X_future)[0])         # expected magnitude
raw_pred = prob * reg_pred                          # smooth gating
```

This is a **soft-gating** approach:

- The classifier outputs a probability (0–1) that today has any demand.
- The regressor outputs a magnitude assuming demand occurs.
- Multiplying them gives the **expected value** under uncertainty.

Why soft instead of hard threshold? An older approach used `if prob < threshold: return 0`. That created a "cliff" — a tiny shift in probability could swing the prediction from 0 to a large number, inflating MAE. Soft gating produces a continuous output that degrades gracefully.

A small spike correction (`if raw_pred > 10: raw_pred *= 1.05`) compensates for the model's tendency to slightly under-predict large values (a known XGBoost regression bias on right-skewed targets).

The result is rounded and clipped to `max(0, ...)` because demand cannot be negative.

5.9 Step 6 — Recursive Forecast (`recursive_forecast_with_meta`)

```
for step in range(1, days_ahead + 1):
    new_date = last_date + pd.Timedelta(days=step)
    for bt in blood_types:
        # 1. Get history up to current step (includes prior predictions)
        # 2. Build features
        # 3. Predict
        # 4. Append prediction to working_df so it becomes input for next step
```

This is **recursive multi-step forecasting**. Each prediction becomes part of the feature set for the next prediction. After 30 iterations there is a full 30-day forecast for every blood type.

Fallback path: If a blood type has fewer than 14 history rows or no trained model, the system falls back to the mean of the last 7 days (or whatever is available). This ensures the API always returns something for every type, even for new hospitals with limited data.

Each row in the output also carries a `prediction_type` tag (`"model"` or `"fallback"`) for transparency.

5.10 Step 7 — Confidence Intervals (Residual-Based)

`build_backtest_comparison`

Splits history at `BACKTEST_CUTOFF_DATE`, retrains features on training data, recursively forecasts the test window, and merges with actuals.

`compute_residual_sd_by_blood_type`

For each blood type:

```
residuals = actual − predicted
sd = residuals.std(ddof=1)
```

This measures the typical magnitude of the model's errors on out-of-sample data.

`prediction_interval_payload`

Constructs the bands:

```
ci_80 = [pred − 1.28 × sd, pred + 1.28 × sd]   # 80% interval (z=1.28)
ci_95 = [pred − 1.96 × sd, pred + 1.96 × sd]   # 95% interval (z=1.96)
```

Lower bounds are clipped at 0. The `interval_status` field flags whether enough data existed to compute the SD.

Why this approach? Bootstrap or quantile regression would be more sophisticated but expensive. Residual-based bands assume errors are roughly normal — this is a pragmatic, transparent method that produces meaningful bands and is easy to defend in a thesis. It also adapts per blood type: types with more volatile errors get wider bands automatically.

5.11 The Endpoints

Endpoint	Returns	Used by
`GET /forecast/{hospital_id}?days=30`	Per-blood-type forecasts with CI bands	Demand tab, Print report
`GET /forecast-total/{hospital_id}?days=30`	Daily summed total predicted demand	Demand tab, Print report
`GET /history-total/{hospital_id}`	Daily summed history (gap-filled)	YoY KPI, Print report
`GET /history/{hospital_id}`	Per-blood-type history (gap-filled)	Trend & streak hooks
`GET /backtest/{hospital_id}`	Actual vs predicted for Oct 2025 + RMSE/MAE/MAPE summary + per-type metrics + CI bands	Backtest mode
`GET /forecast-map`	All hospitals with risk levels for map view	Forecast map

Each endpoint is wrapped in a try/except that returns `{"error": ...}` instead of raising — so the frontend can degrade gracefully.

5.12 Backtest Endpoint in Detail

The `/backtest/{hospital_id}` endpoint is the **scientific validation surface** of the system:

1. Load full history, fill gaps.
2. Call `build_backtest_comparison` to retrain on data up to Sep 30 2025 and forecast Oct 1–31 2025.
3. Merge predictions with actuals on `(date, blood_type)`.
4. Build a **lag-1 baseline** — `lag_1_pred[d] = blood_requests[d-1]`. This is the "naïve" benchmark: "tomorrow will be like today."
5. Compute RMSE, MAE, MAPE for both the model and the baseline. The model only "wins" if it beats lag-1.
6. Compute residual SD per blood type.
7. Attach 80% / 95% CI bands to every prediction row.
8. Return everything in one JSON payload.

This is what the thesis demonstrates as proof of model quality — the comparison against lag-1 shows the model is genuinely learning patterns, not just memorizing.

5.13 Risk Level Helpers (Used by `/forecast-map`)

```
def compute_days_cover(stock, total_predicted_demand, days=30):
    return stock / (total_predicted_demand / days)

def compute_risk_level(days_cover):
    if days_cover <= 3:  return "critical"
    if days_cover <= 7:  return "warning"
    return "safe"
```

This translates raw forecast numbers into operational risk labels. A hospital with 20 units of stock and a forecast of 60 units over 30 days has 10 days of cover — "safe". With only 5 units it has 2.5 days — "critical". This is what the map dashboard color-codes.

---

6. End-to-End Data Flow Summary

A full request lifecycle when a user opens the Demand Forecasting tab:

1. **Browser** → React mounts `DemandForecastingTab` with `hospitalId`.
2. **React** → Fires four parallel axios requests via `/api/...`.
3. **Node/Express** → Proxies the calls to FastAPI (or FastAPI is hit directly via the configured `api` instance).
4. **FastAPI** → For each endpoint:
    - Queries PostgreSQL via SQLAlchemy.
    - Fills missing dates.
    - For forecast endpoints: builds features → loads XGBoost models → runs recursive forecast → computes CI bands.
5. **FastAPI** → Returns JSON.
6. **React** → Stores results in state, derives KPIs and chart data via `useMemo`, and renders charts with Recharts.
7. **User** → Switches to **Forecast Report** tab → identical fetches → printable layout.
8. **User** → Clicks **Print** → browser print dialog → PDF.

For an **Import**:

1. User uploads CSV/XLSX in `ImportPage`.
2. `multipart/form-data` POST to `/api/import/requests`.
3. `importRoutes.js` validates each row, runs a transactional bulk insert.
4. PostgreSQL `requests` table grows.
5. Next time `/forecast/...` is called, the new rows are included in `load_center_history`, expanding the model's input window and improving feature quality.

Appendix A — Why These Specific Charts and KPIs Were Chosen

Element	Decision Driver
30-day horizon	Aligns with typical hospital procurement cycles
Per-blood-type breakdown	Each type has distinct supply chains and shelf lives
7-day vs 30-day top type	Captures both short-term urgency and monthly planning
YoY KPI	Sanity-check against historical reality
Streak alerts (≥3 days)	Filters noise; sustained pressure is what requires action
80% / 95% CI	Lets planners reason about both typical and worst-case scenarios
Lag-1 baseline in backtest	Industry-standard naïve benchmark; proves the model is non-trivial
Day-of-week chart	Reveals weekly seasonality for staffing decisions

Appendix B — Why the Two-Stage Model + Soft Gating

Blood request data per blood type is **zero-inflated**: many days have no requests. A single regressor would either:

- Predict near-zero everywhere (biased to majority class), or
- Predict moderate values everywhere (large MAE on zero days).

Splitting into classifier (occurrence) + regressor (magnitude) lets each sub-model specialize. Multiplying their outputs (`prob × reg_pred`) gives the mathematical expectation, which is a smooth, well-calibrated point forecast — and avoids the "cliff edge" of a hard threshold.

End of document.